

**PROJETO INOVARU**

Hackathon de Créditos Universitários — 2026/01

# Especificação Técnica

API InovaRU — Integração do Aplicativo Móvel

| Campo                  | Informação                                         |
|------------------------|----------------------------------------------------|
| Versão                 | 2.0 (final) — evolui a versão preliminar 1.0       |
| Data                   | 03/07/2026                                         |
| Emissor                | FUMP — Fundação Universitária Mendes Pimentel      |
| Backend                | Node.js + Express.js · Banco de dados SQL Server   |
| Gateway de pagamento   | MercadoPago (PIX)                                  |
| Contato técnico FUMP   | Leonardo Cunha — leonardo.cunha@fump.ufmg.br       |
| Contato DACompSI       | Raúl Simioni — raulsimioni@dcc.ufmg.br             |
| Documento complementar | Documento Conceitual (fluxo de telas e wireframes) |

## Sumário

|                                                             |    |
|-------------------------------------------------------------|----|
| Sumário .....                                               | 2  |
| 1. Visão geral .....                                        | 3  |
| 1.1. Componentes .....                                      | 3  |
| 1.2. O que mudou em relação à versão preliminar (1.0) ..... | 3  |
| 2. Arquitetura e fluxo principal .....                      | 3  |
| 3. Convenções da API .....                                  | 4  |
| 4. Ambientes e Base URL .....                               | 5  |
| 4.1. Simulando a API localmente (mock servers) .....        | 5  |
| 5. Autenticação .....                                       | 6  |
| 6. Modelo de dados (dicionário) .....                       | 6  |
| 6.1. Consumidor .....                                       | 6  |
| 6.2. Saldo .....                                            | 6  |
| 6.3. Pagamento .....                                        | 6  |
| 6.4. Recarga (item de histórico) .....                      | 7  |
| 6.5. Refeição (item de histórico) .....                     | 7  |
| 6.6. Filial (Restaurante Universitário) .....               | 7  |
| 6.7. Usuário (retorno do login) .....                       | 7  |
| 6.8. Paginação (envelope comum de listagens) .....          | 7  |
| 7. Referência de endpoints .....                            | 8  |
| 7.1. Autenticar usuário .....                               | 8  |
| 7.2. Consultar saldo e dados do consumidor .....            | 8  |
| 7.3. Criar pagamento PIX (solicitar recarga) .....          | 9  |
| 7.4. Consultar status do pagamento (polling) .....          | 9  |
| 7.5. Histórico de recargas .....                            | 9  |
| 7.6. Histórico de refeições .....                           | 10 |
| 8. Máquina de estados do pagamento .....                    | 11 |
| 9. Códigos de erro .....                                    | 11 |
| 10. Segurança e autorização .....                           | 11 |
| 10.1. Autorização por ownership .....                       | 11 |
| 10.2. Rate limiting .....                                   | 12 |
| 10.3. Comunicação .....                                     | 12 |
| 11. Recomendações de implementação (app) .....              | 12 |
| 11.1. Autenticação e segurança .....                        | 12 |
| 11.2. Fluxo de recarga PIX .....                            | 12 |
| 11.3. Paginação, fuso e tecnologias .....                   | 12 |
| 12. Maturidade e evolução do contrato .....                 | 13 |
| 13. Glossário .....                                         | 13 |
| Anexos .....                                                | 13 |
| Anexo A — Restaurantes Universitários .....                 | 13 |
| Anexo B — Estados do pagamento .....                        | 14 |
| Anexo C — Situação do consumidor .....                      | 14 |
| Anexo D — Limites de requisição .....                       | 14 |

## 1. Visão geral

A FUMP disponibiliza uma API RESTful para que o aplicativo móvel do Hackathon InovaRU realize a autenticação de usuários, a consulta de saldo e dados do consumidor, a geração de pagamentos PIX e a consulta de históricos de recargas e refeições nos Restaurantes Universitários (RUs) da UFMG. O aplicativo é desenvolvido para Android e, ao final do hackathon, entregue à FUMP para implantação no ambiente institucional.

**Papel do aplicativo.** O crédito no sistema RU é acionado pelo **webhook** do MercadoPago diretamente na API da FUMP — **o aplicativo não credita saldo**. O aplicativo apenas inicia o pagamento, exibe o QR Code PIX e faz **polling** do status até receber a confirmação.

### 1.1. Componentes

| Componente                 | Responsabilidade                                                                           |
|----------------------------|--------------------------------------------------------------------------------------------|
| Aplicativo móvel (Android) | Interface do usuário; consome a API via HTTPS; exibe QR Code e faz polling do pagamento.   |
| API FUMP (Node.js/Express) | Autenticação JWT, regras de negócio, integração com o MercadoPago e com a base SQL Server. |
| SQL Server                 | Base institucional: consumidores, saldos, recargas e refeições.                            |
| MercadoPago                | Gateway PIX: gera o QR Code e notifica a API por webhook quando o pagamento é aprovado.    |

### 1.2. O que mudou em relação à versão preliminar (1.0)

Esta versão consolida e detalha o contrato já esboçado na versão preliminar 1.0 (25/04/2026). O conjunto de endpoints e o comportamento de negócio permanecem os mesmos; o foco desta revisão é dar clareza e completude suficientes para a implementação e a validação da integração.

| Item                                         | Situação nesta versão                                                                                                                        |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Diagrama de sequência do fluxo principal     | Adicionado (substitui o esquema textual da 1.0).                                                                                             |
| Dicionário de dados (entidades e campos)     | Adicionado.                                                                                                                                  |
| Máquina de estados do pagamento              | Adicionada, com transições e ação esperada do app.                                                                                           |
| Exemplos de requisição/resposta por endpoint | Detalhados, com tabelas de campos.                                                                                                           |
| Catálogo de erros e envelope padrão          | Detalhado.                                                                                                                                   |
| Recomendações de implementação do app        | Detalhadas (Keystore, polling, paginação, fuso).                                                                                             |
| Recomendação de polling                      | Unificada em backoff exponencial com jitter (a 1.0 citava também um intervalo fixo de 5 s na seção 4.3).                                     |
| Tabelas de apoio (RUs, status, limites)      | Consolidadas como anexos.                                                                                                                    |
| Ambiente de testes                           | Redefinido: desenvolvimento contra mock local do contrato (Mockoon, JSON Server ou Postman — ver seção 4.1). Produção permanece “a definir”. |

## 2. Arquitetura e fluxo principal

O diagrama abaixo resume a jornada completa de uma recarga bem-sucedida: autenticação, consulta de saldo, criação do pagamento PIX, confirmação por webhook e acompanhamento por polling até o crédito do saldo.



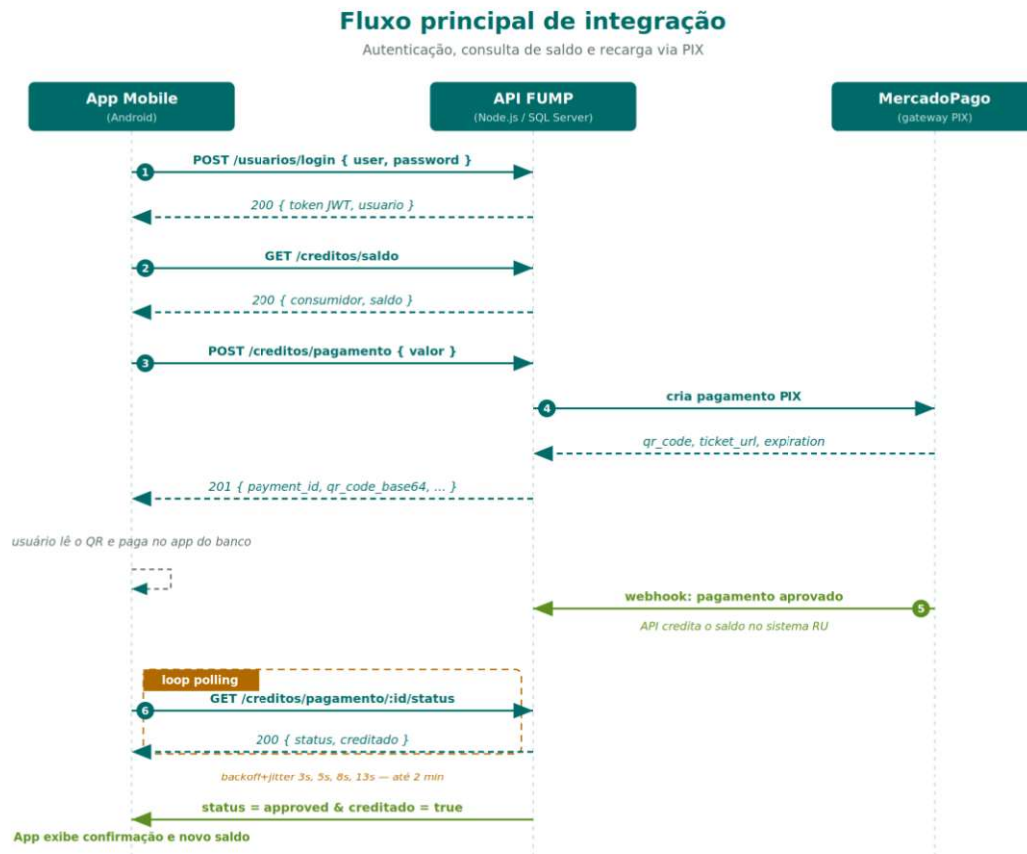


Figura 1 — Fluxo principal de integração (caminho de sucesso).

**Observação sobre o polling.** Após criar o pagamento, o app consulta periodicamente o status até que ele deixe de ser `pending` ou até atingir 2 minutos (o que ocorrer primeiro). O crédito do saldo é feito pela API ao receber o webhook de aprovação; o campo `creditado` indica quando o saldo já está disponível para uso nos RUs.

### 3. Convenções da API

| Aspecto            | Convenção                                                               |
|--------------------|-------------------------------------------------------------------------|
| Estilo             | RESTful sobre HTTP; recursos no plural (ex.: /creditos/recargas).       |
| Protocolo          | HTTPS obrigatório (TLS 1.2+). Requisições HTTP são rejeitadas.          |
| Formato            | JSON em requisição e resposta (Content-Type: application/json).         |
| Codificação        | UTF-8.                                                                  |
| Datas e horas      | ISO 8601 com fuso de Brasília (-03:00). Ex.: 2026-04-27T10:30:00-03:00. |
| Valores monetários | Decimais em reais (R\$), ponto como separador decimal. Ex.: 45.50.      |
| Autenticação       | JWT no header Authorization: Bearer <token> (exceto no login).          |
| Envelope de erro   | Objeto JSON com o campo message (ver seção 9).                          |
| Paginação          | Listagens retornam o objeto pagination (ver seção 6.8).                 |

## 4. Ambientes e Base URL

Durante o hackathon, a FUMP **não disponibilizará um ambiente de homologação**. O desenvolvimento e a demonstração do aplicativo devem ser feitos contra um **mock local** — um servidor simulado, criado pela própria equipe, que reproduz fielmente a especificação descrita neste documento (ver seção 4.1).

| Ambiente                    | Base URL                                                        | Situação                                           |
|-----------------------------|-----------------------------------------------------------------|----------------------------------------------------|
| Desenvolvimento (hackathon) | Mock local da equipe (ex.: <code>http://localhost:3001</code> ) | Simulado a partir deste documento — ver seção 4.1. |

Todas as rotas descritas neste documento são relativas à Base URL do ambiente.

Exemplo de composição: `https://<base-url>/creditos/saldo`.

*O certificado SSL válido será implementado no ambiente de produção.*

### 4.1. Simulando a API localmente (mock servers)

Os exemplos de requisição e resposta das seções 5 e 7 podem ser copiados diretamente para a ferramenta de mock escolhida pela equipe. Sugestões consolidadas:

| Ferramenta           | Tipo                              | Indicação                                                                                                                                                                                                |
|----------------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mockoon              | Aplicativo desktop gratuito (GUI) | Caminho recomendado: rotas criadas visualmente, respostas sequenciais/por regras para simular a transição <code>pending</code> → <code>approved</code> do pagamento, além de latência e códigos de erro. |
| JSON Server          | Pacote Node.js (npm)              | Mock leve por linha de comando a partir de um <code>db.json</code> ; prático para as listagens paginadas (recargas e refeições).                                                                         |
| Postman Mock Servers | Nuvem (plano gratuito)            | Mock hospedado a partir de uma collection; útil quando a equipe quer compartilhar a mesma URL de teste.                                                                                                  |

Boas práticas ao usar o mock:

- **Base URL configurável.** Deixe a Base URL parametrizada no app (ex.: `BuildConfig / product flavors` no Android), para alternar entre o mock e a produção sem alterar código.
- **Simule também os erros e estados.** Cubra 401, 422 e 429 e a evolução do pagamento (`pending` → `approved` / `expired`) — o tratamento de erros é critério de avaliação do edital (item 6.2).
- **HTTP no mock é aceitável.** O requisito de HTTPS (seção 3) vale para a produção. Em desenvolvimento, libere o tráfego HTTP do app (`network security config` ou `usesCleartextTraffic`), pois o Android bloqueia `cleartext` por padrão.
- **Emulador Android.** Para acessar o mock rodando na máquina de desenvolvimento, use `http://10.0.2.2:<porta>` — o `localhost` do emulador aponta para o próprio dispositivo virtual.
- **Token de teste.** No mock, o JWT pode ser uma string fixa qualquer; o que importa é o app montar corretamente o header `Authorization: Bearer <token>`.

## 5. Autenticação

Todas as requisições, exceto o login, devem incluir o token JWT no header HTTP:

```
Authorization: Bearer <jwt>
```

- O token é obtido no endpoint `POST /usuarios/login` e tem validade limitada.
- Quando o token expira, a API responde `401`; o app deve solicitar novo login ao usuário.
- O CPF do usuário é extraído automaticamente do token — **não é necessário enviar CPF** no body ou na URL.
- A senha usada no login é a mesma do sistema institucional da FUMP (portal do aluno/colaborador). O app **não cria contas** — apenas autentica usuários já cadastrados.

## 6. Modelo de dados (dicionário)

As entidades abaixo descrevem os objetos trafegados pela API. Os endpoints da seção 7 referenciam estes campos.

### 6.1. Consumidor

| Campo           | Tipo   | Descrição                                                                        |
|-----------------|--------|----------------------------------------------------------------------------------|
| nome            | string | Nome completo do consumidor.                                                     |
| tipo_consumidor | object | Categoria: { codigo, descricao } (ex.: 01 — Estudante Regular).                  |
| centro_custo    | object | Vínculo acadêmico: { tipo, descricao } (ex.: Graduação — Ciência da Computação). |
| situacao        | string | "A" = Ativo, "I" = Inativo, "B" = Bloqueado.                                     |

### 6.2. Saldo

| Campo              | Tipo    | Descrição                                     |
|--------------------|---------|-----------------------------------------------|
| credito_disponivel | decimal | Saldo em R\$ disponível para uso nos RUs.     |
| limite_recarga     | decimal | Valor máximo que pode ser mantido como saldo. |

### 6.3. Pagamento

| Campo          | Tipo   | Descrição                                                          |
|----------------|--------|--------------------------------------------------------------------|
| payment_id     | int    | Identificador do pagamento no MercadoPago; chave usada no polling. |
| status         | string | Estado do pagamento (ver máquina de estados, seção 8).             |
| status_detail  | string | Detalhe do status, quando disponível (ex.: accredited).            |
| qr_code        | string | Código PIX copia-e-cola.                                           |
| qr_code_base64 | string | Imagem PNG do QR Code em Base64 (para exibição no app).            |
| ticket_url     | string | URL do ticket de pagamento (fallback).                             |
| expiration     | string | Data/hora de expiração do PIX (ISO 8601, -03:00).                  |



| Campo     | Tipo    | Descrição                                           |
|-----------|---------|-----------------------------------------------------|
| creditado | boolean | true quando o saldo já foi creditado no sistema RU. |

#### 6.4. Recarga (item de histórico)

| Campo     | Tipo    | Descrição                                |
|-----------|---------|------------------------------------------|
| id        | int     | Identificador da recarga.                |
| valor     | decimal | Valor recarregado (R\$).                 |
| metodo    | string  | Meio de pagamento (ex.: pix).            |
| status    | string  | Situação da recarga (ex.: aprovado).     |
| data_hora | string  | Data/hora da recarga (ISO 8601, -03:00). |

#### 6.5. Refeição (item de histórico)

| Campo           | Tipo    | Descrição                                    |
|-----------------|---------|----------------------------------------------|
| data_hora       | string  | Data/hora da refeição (ISO 8601, -03:00).    |
| filial          | object  | RU onde ocorreu a venda: { código, nome }.   |
| quantidade      | int     | Quantidade de refeições na venda.            |
| valor_total     | decimal | Valor pago na venda (R\$).                   |
| gratuidade      | boolean | true se a refeição foi gratuita.             |
| tipo_consumidor | string  | Categoria do consumidor no momento da venda. |

#### 6.6. Filial (Restaurante Universitário)

| Campo  | Tipo   | Descrição                                 |
|--------|--------|-------------------------------------------|
| codigo | string | Código do RU (0001 a 0005 — ver Anexo A). |
| nome   | string | Nome do RU.                               |

#### 6.7. Usuário (retorno do login)

| Campo         | Tipo   | Descrição                                       |
|---------------|--------|-------------------------------------------------|
| nome          | string | Nome do usuário autenticado.                    |
| email         | string | E-mail institucional.                           |
| isAluno       | int    | 1 se o usuário é aluno; caso contrário 0.       |
| isColaborador | int    | 1 se o usuário é colaborador; caso contrário 0. |

#### 6.8. Paginação (envelope comum de listagens)

| Campo       | Tipo | Descrição                   |
|-------------|------|-----------------------------|
| total       | int  | Total de itens disponíveis. |
| currentPage | int  | Página atual.               |
| perPage     | int  | Itens por página.           |
| lastPage    | int  | Última página disponível.   |

## 7. Referência de endpoints

Salvo o login, todos os endpoints exigem autenticação JWT (seção 5). O CPF é extraído do token; nenhuma rota recebe CPF no body ou na URL.

### 7.1. Autenticar usuário

#### POST /usuarios/login

Autentica o usuário com CPF e senha institucional e retorna o token JWT.

##### Requisição

```
{
  "user": "12345678901",
  "password": "senha_do_usuario"
}
```

| Campo    | Tipo   | Obrigatório | Descrição                                     |
|----------|--------|-------------|-----------------------------------------------|
| user     | string | Sim         | CPF do usuário (somente números, 11 dígitos). |
| password | string | Sim         | Senha do sistema institucional da FUMP.       |

##### Resposta 200 — sucesso

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "usuario": {
    "nome": "JOÃO DA SILVA",
    "email": "joao@ufmg.br",
    "isAluno": 1,
    "isColaborador": 0
  }
}
```

##### Resposta 401 — credenciais inválidas

```
{ "message": "Usuário ou senha inválidos" }
```

### 7.2. Consultar saldo e dados do consumidor

#### GET /creditos/saldo

Retorna os dados do consumidor e o saldo vinculados ao token.

##### Resposta 200 — sucesso

```
{
  "consumidor": {
    "nome": "JOÃO DA SILVA",
    "tipo_consumidor": { "codigo": "01", "descricao": "ESTUDANTE REGULAR" },
    "centro_custo": { "tipo": "Graduação", "descricao": "CIENCIA DA COMPUTACAO" },
    "situacao": "A"
  },
  "saldo": {
    "credito_disponivel": 45.50,
    "limite_recarga": 500.00
  }
}
```

Campos detalhados nas seções 6.1 (Consumidor) e 6.2 (Saldo).



**Resposta 404 — consumidor não encontrado**

```
{ "message": "Consumidor não encontrado ou inativo." }
```

### 7.3. Criar pagamento PIX (solicitar recarga)

**POST** `/creditos/pagamento`

Cria um pagamento PIX no MercadoPago e retorna os dados do QR Code para exibição no app.

**Requisição**

```
{ "valor": 50.00 }
```

| Campo | Tipo    | Obrigatório | Validação                            |
|-------|---------|-------------|--------------------------------------|
| valor | decimal | Sim         | Mínimo R\$ 5,00 / Máximo R\$ 500,00. |

**Resposta 201 — pagamento criado**

```
{
  "payment_id": 123456789,
  "status": "pending",
  "qr_code": "00020126580014br.gov.bcb.pix...",
  "qr_code_base64": "iVBORw0KGgoAAAANSUHEug...",
  "ticket_url": "https://www.mercadopago.com.br/payments/123456789/ticket",
  "expiration": "2026-04-27T11:00:00-03:00"
}
```

Campos detalhados na seção 6.3 (Pagamento). O status inicial é sempre `pending`.

**Resposta 422 — valor fora do limite**

```
{ "message": "Valor fora do limite permitido. Mínimo: R$ 5,00, Máximo: R$ 500,00" }
```

### 7.4. Consultar status do pagamento (polling)

**GET** `/creditos/pagamento/:paymentId/status`

Verifica o estado de um pagamento criado pelo usuário. Deve ser consultado por polling após a exibição do QR Code (ver recomendações na seção 11).

| Parâmetro | Local | Tipo | Descrição                                               |
|-----------|-------|------|---------------------------------------------------------|
| paymentId | path  | int  | ID retornado em POST <code>/creditos/pagamento</code> . |

**Resposta 200 — estado atual**

```
{
  "payment_id": 123456789,
  "status": "approved",
  "status_detail": "accredited",
  "creditado": true
}
```

Valores possíveis de status e a ação esperada do app estão na seção 8.

### 7.5. Histórico de recargas

**GET** `/creditos/recargas`

Retorna o histórico de recargas de crédito realizadas pelo app, com paginação e filtros opcionais por data.

**Parâmetros de consulta (query)**

| Parâmetro  | Tipo   | Obrigatório | Default | Descrição                        |
|------------|--------|-------------|---------|----------------------------------|
| page       | int    | Não         | 1       | Página.                          |
| perPage    | int    | Não         | 20      | Itens por página (máx. 100).     |
| dataInicio | string | Não         | —       | Filtro data início (YYYY-MM-DD). |
| dataFim    | string | Não         | —       | Filtro data fim (YYYY-MM-DD).    |

**Resposta 200 — lista paginada**

```
{
  "data": [
    { "id": 1, "valor": 50.00, "metodo": "pix", "status": "aprovado",
      "data_hora": "2026-04-27T10:30:00-03:00" },
    { "id": 2, "valor": 30.00, "metodo": "pix", "status": "aprovado",
      "data_hora": "2026-04-20T14:15:00-03:00" }
  ],
  "pagination": { "total": 15, "currentPage": 1, "perPage": 20, "lastPage": 1 }
}
```

Campos do item na seção 6.4 (Recarga); envelope de paginação na seção 6.8.

**7.6. Histórico de refeições****GET /creditos/refeicoes**

Retorna o histórico de refeições/consumos do usuário nos RUs, com paginação e filtros opcionais.

**Parâmetros de consulta (query)**

| Parâmetro  | Tipo   | Obrigatório | Default | Descrição                        |
|------------|--------|-------------|---------|----------------------------------|
| page       | int    | Não         | 1       | Página.                          |
| perPage    | int    | Não         | 20      | Itens por página (máx. 100).     |
| dataInicio | string | Não         | —       | Filtro data início (YYYY-MM-DD). |
| dataFim    | string | Não         | —       | Filtro data fim (YYYY-MM-DD).    |
| filial     | string | Não         | —       | Código do RU (ver Anexo A).      |

**Resposta 200 — lista paginada**

```
{
  "data": [
    {
      "data_hora": "2026-04-25T12:05:33-03:00",
      "filial": { "codigo": "0003", "nome": "RU Setorial 1" },
      "quantidade": 1,
      "valor_total": 2.40,
      "gratuidade": false,
      "tipo_consumidor": "ESTUDANTE REGULAR"
    }
  ],
  "pagination": { "total": 142, "currentPage": 1, "perPage": 20, "lastPage": 8 }
}
```

Campos do item na seção 6.5 (Refeição); envelope de paginação na seção 6.8.

## 8. Máquina de estados do pagamento

O campo `status` retornado no polling (seção 7.4) evolui conforme o pagamento. O app deve reagir da seguinte forma:

| Status    | Significado                | Ação do app                                   |
|-----------|----------------------------|-----------------------------------------------|
| pending   | Aguardando pagamento.      | Continuar o polling.                          |
| approved  | Pagamento aprovado.        | Parar o polling; exibir sucesso e novo saldo. |
| rejected  | Pagamento rejeitado.       | Parar o polling; informar falha.              |
| cancelled | Pagamento cancelado.       | Parar o polling; permitir nova tentativa.     |
| expired   | PIX expirou sem pagamento. | Parar o polling; permitir nova tentativa.     |

**Nota.** O MercadoPago pode retornar `cancelled` ou `expired` em situações de expiração. O app deve tratar ambos da mesma forma: informar o usuário e oferecer a opção de gerar um novo pagamento. Quando `status = approved` e `creditado = true`, o saldo já está disponível para uso nos RUs.

## 9. Códigos de erro

As respostas de erro seguem um envelope único com o campo `message`:

```
{ "message": "Descrição legível do erro." }
```

| HTTP | Significado                                 | Ação recomendada no app                    |
|------|---------------------------------------------|--------------------------------------------|
| 400  | Requisição malformada.                      | Verificar o payload enviado.               |
| 401  | Token ausente, inválido ou expirado.        | Redirecionar para a tela de login.         |
| 403  | Sem permissão para o recurso.               | Informar o usuário (recurso de outro CPF). |
| 404  | Consumidor não encontrado.                  | Informar que o CPF não está cadastrado.    |
| 422  | Dados inválidos (ex.: valor fora do range). | Exibir mensagem de validação.              |
| 429  | Limite de requisições excedido.             | Aguardar (Retry-After) e tentar novamente. |
| 500  | Erro interno do servidor.                   | Exibir mensagem genérica de erro.          |

## 10. Segurança e autorização

### 10.1. Autorização por ownership

A API garante que o usuário autenticado só acessa recursos vinculados ao seu próprio CPF:

- `GET /creditos/saldo` retorna apenas os dados do CPF do token.
- `GET /creditos/pagamento/:paymentId/status` retorna apenas pagamentos criados pelo usuário autenticado; consultar pagamentos de terceiros resulta em `403`.
- `GET /creditos/recargas` e `GET /creditos/refeicoes` retornam apenas dados do CPF do token.



## 10.2. Rate limiting

| Endpoint                 | Limite               | Observação                      |
|--------------------------|----------------------|---------------------------------|
| POST /usuarios/login     | 5 req/min por IP     | Proteção contra brute force.    |
| POST /creditos/pagamento | 10 req/min por token | Evita criação excessiva de PIX. |
| GET (consultas)          | 60 req/min por token | Uso geral.                      |

Ao atingir o limite, a API responde 429 com o header `Retry-After` (segundos até a próxima requisição permitida). O app deve tratar o 429 com mensagem amigável, por exemplo: “Muitas tentativas, aguarde um momento”.

## 10.3. Comunicação

- HTTPS obrigatório (TLS 1.2+). Requisições HTTP são rejeitadas.
- Certificado SSL válido será implementado no ambiente de produção.

# 11. Recomendações de implementação (app)

## 11.1. Autenticação e segurança

- **Nunca armazenar o token JWT em texto puro** (por exemplo, em `SharedPreferences` padrão). Usar `Android Keystore` ou `EncryptedSharedPreferences`.
- **Nunca persistir a senha do usuário** no dispositivo, em hipótese alguma.
- Tratar token expirado (401) redirecionando para a tela de login.
- Usar exclusivamente o header `Authorization: Bearer <token>` para autenticação.

## 11.2. Fluxo de recarga PIX

- Chamar `POST /creditos/pagamento` com o valor desejado.
- Exibir o QR Code decodificando `qr_code_base64` (Base64 → PNG).
- Oferecer o botão “Copiar código PIX” usando o campo `qr_code` (copia-e-cola).
- Iniciar o polling em `GET /creditos/pagamento/:id/status` com backoff exponencial e jitter: 3s, 5s, 8s, 13s... acrescidos de variação aleatória de  $\pm 1s$ . Isso evita que vários usuários simultâneos sincronizem as requisições e sobrecarreguem a API.
- Parar o polling quando `status != pending` ou após 2 minutos (o que ocorrer primeiro).
- Em `approved` com `creditado: true`, exibir a confirmação e atualizar o saldo.
- Se o polling expirar (2 min) ou o status for `cancelled/expired`, exibir mensagem e oferecer “Tentar novamente”.

## 11.3. Paginação, fuso e tecnologias

- Listagens retornam o objeto `pagination`; implementar scroll infinito ou botão “carregar mais”.
- Todas as datas vêm em ISO 8601 com fuso de Brasília (-03:00); tratar a conversão para o fuso do dispositivo quando necessário.
- HTTP: `Retrofit` (Java/Kotlin) ou `Ktor` (Kotlin).
- QR Code: decodificar Base64 para `Bitmap` e exibir em `ImageView`.
- Cache offline de histórico (opcional): `Room`.

- Baixa conectividade: exibir estados de carregamento e erro claros, permitir novas tentativas e evitar bloqueios da interface durante o polling.

## 12. Maturidade e evolução do contrato

Este documento adota uma abordagem de contract-first: define de forma estável a interface entre o aplicativo e a API antes da conclusão do desenvolvimento do backend, para que as equipes possam projetar e implementar o app com previsibilidade.

- O conjunto de endpoints, campos e regras de negócio aqui descrito é a referência oficial para o desenvolvimento durante o hackathon.
- Ajustes pontuais que se mostrem necessários na implementação serão comunicados pela organização, com a devida antecedência.
- Durante o hackathon, o desenvolvimento é feito contra mocks locais que simulam este contrato (seção 4.1). A Base URL de produção e o certificado SSL serão definidos na fase de implantação institucional.

## 13. Glossário

| Termo          | Descrição                                                                                       |
|----------------|-------------------------------------------------------------------------------------------------|
| JWT            | JSON Web Token; credencial assinada usada na autenticação via header Authorization.             |
| PIX            | Meio de pagamento instantâneo brasileiro; aqui gerado via MercadoPago.                          |
| Webhook        | Notificação enviada pelo MercadoPago à API quando o pagamento muda de estado.                   |
| Polling        | Consulta periódica ao status do pagamento até a confirmação ou o tempo limite.                  |
| Backoff/Jitter | Intervalos crescentes com variação aleatória entre requisições, para reduzir picos simultâneos. |
| Ownership      | Regra de autorização que restringe o acesso aos recursos do próprio CPF.                        |
| RU             | Restaurante Universitário.                                                                      |
| Consumidor     | Pessoa vinculada ao sistema RU identificada pelo CPF.                                           |

## Anexos

### Anexo A — Restaurantes Universitários

| Código | Nome             | Localização                                 |
|--------|------------------|---------------------------------------------|
| 0001   | RU Saúde/Direito | Campus Saúde, Belo Horizonte.               |
| 0002   | RU Setorial 2    | Campus Pampulha, Belo Horizonte.            |
| 0003   | RU Setorial 1    | Campus Pampulha, Belo Horizonte.            |
| 0004   | RU ICA           | Campus Montes Claros.                       |
| 0005   | RU HRTN          | Hospital Risoleta T. Neves, Belo Horizonte. |

## Anexo B — Estados do pagamento

| Status    | Terminal? | Significado                                                  |
|-----------|-----------|--------------------------------------------------------------|
| pending   | Não       | Aguardando pagamento.                                        |
| approved  | Sim       | Pagamento aprovado; crédito confirmado pelo campo creditado. |
| rejected  | Sim       | Pagamento rejeitado.                                         |
| cancelled | Sim       | Pagamento cancelado.                                         |
| expired   | Sim       | PIX expirou sem pagamento.                                   |

## Anexo C — Situação do consumidor

| Código | Situação  | Efeito                                             |
|--------|-----------|----------------------------------------------------|
| A      | Ativo     | Uso normal do saldo e das recargas.                |
| I      | Inativo   | Consulta pode retornar 404; sem operações.         |
| B      | Bloqueado | Recarga desabilitada no app, com aviso ao usuário. |

## Anexo D — Limites de requisição

| Endpoint                 | Limite     | Escopo     |
|--------------------------|------------|------------|
| POST /usuarios/login     | 5 req/min  | Por IP.    |
| POST /creditos/pagamento | 10 req/min | Por token. |
| GET (consultas)          | 60 req/min | Por token. |